

Introduction au développement frontend moderne

Objectifs de cette section

Dans cette introduction, nous allons poser les bases du **développement frontend moderne**, comprendre l'évolution du rôle du navigateur, et introduire les concepts fondamentaux nécessaires pour aborder les frameworks modernes.

À la fin de cette section, vous serez capable de :

- Comprendre ce qu'est le **DOM** et pourquoi il est essentiel dans une interface web.
- Identifier les **limitations du DOM classique** et la montée des composants.
- Distinguer les approches **Web Components** et **frameworks comme React**.
- Vous préparer à **concevoir une application frontend structurée** dans les prochaines sections.

1. Qu'est-ce que le DOM et pourquoi est-il important ?

Le **DOM (Document Object Model)** est une **représentation arborescente** d'une page web, manipulable dynamiquement avec JavaScript. Il permet de :

- Modifier dynamiquement le contenu, la structure et le style d'une page.
- Gérer les **événements utilisateurs** (clic, saisie, survol, etc.).
- Créer une **expérience interactive** sans recharger la page.

Exemple : Afficher/masquer une section

```
<!-- index.html -->
<button id="toggle">Afficher plus</button>
<div id="details" style="display: none;">Contenu supplémentaire</div>
```

```
// script.js
document.getElementById("toggle").addEventListener("click", () => {
  const target = document.getElementById("details");
  const isHidden = target.style.display === "none";
  target.style.display = isHidden ? "block" : "none";
  document.getElementById("toggle").textContent = isHidden ? "Masquer" : "Afficher plus";
});
```

2. Limites du DOM et naissance des composants

À mesure qu'une interface web devient plus complexe, manipuler le DOM directement pose plusieurs problèmes :

- **Difficulté de réutilisation** : on recopie du code HTML au lieu de le factoriser.
- **Effets de bord** : plusieurs scripts peuvent modifier les mêmes éléments.
- **Lisibilité** : le code devient rapidement difficile à maintenir.

Vers une approche modulaire

Un **composant** est une brique autonome de l'interface utilisateur, qui encapsule :

- Un **template HTML**.
- Du **style** spécifique.
- Une **logique JavaScript**.

3. Web Components : des composants natifs

Les **Web Components** sont une technologie standard du navigateur permettant de créer des composants personnalisés, sans framework externe. Ils reposent sur quatre briques principales :

- **Custom Elements** : créer de nouvelles balises HTML.
- **Shadow DOM** : encapsuler le style et l'arborescence du composant.
- **HTML Templates** : définir des structures HTML réutilisables.
- **Slots** : insérer du contenu dans un composant depuis l'extérieur.

Exemple : Création d'un élément personnalisé

```
<!-- index.html -->
<hello-world></hello-world>
<script type="module" src="hello-world.js"></script>
```

```
// hello-world.js
class HelloWorld extends HTMLElement {
  connectedCallback() {
    this.innerHTML = `<p>Hello, world!</p>`;
  }
}

customElements.define("hello-world", HelloWorld);
```

4. Exercice de mise en pratique

Consignes

1. Créez une **page HTML** contenant :
 - Un **bouton** affichant "Afficher la tâche".
 - Une **zone** contenant une tâche simple (ex. : "Acheter du pain"), cachée par défaut.
2. Ajoutez un **fichier JavaScript** qui :
 - Affiche ou masque cette tâche au clic sur le bouton.
 - Change le texte du bouton en fonction de l'état ("Afficher la tâche" / "Masquer la tâche").

Organisation attendue

```
/exercices
├── 01-toggle-tache
│   ├── index.html
│   └── script.js
```